Equipment Fault Diagnosis System in B-Prolog

1. For this assignment, I built a small production system in B-Prolog to diagnose faults in factory rotating machinery based on sensor data. I found that B-Prolog's dynamic predicates are actually really handy for simulating both Short-Term Memory (STM) and Long-Term Memory (LTM).

Following the standard setup of production systems, here is how the different parts of my architecture work together:

1) Observations to Short-Term Memory: The current sensor readings are basically converted into facts and saved right into the STM.
2) Long-Term Memory: The LTM holds all the production rules that were generated from previous diagnostic cases.
3) Inference Engine: This component does the heavy lifting. It compares the facts currently sitting in the STM against the available rules in the LTM. If multiple rules match up, it handles the conflict resolution and fires the best one.
4) Output: Any newly inferred facts get pushed back into the STM so the system can keep doing multi-step forward chaining. In the end, it outputs the final diagnosis, a confidence score, and a breakdown of the reasoning steps.

2. Task Implementations

Task 1: Representing Production Rules

I set up the production rules in the LTM as structured Prolog facts. The data structure looks like this: `rule(RuleID, ConditionsList, Conclusion, Confidence, Support)`. It basically packs everything we need into one place—the rule ID, the conditions, the conclusion, the confidence score, and the support count. So, for example, if the temperature and vibration are both high, the rule triggers and concludes that there's a bearing failure.

Task 2: Generating Long-Term Memory through Inductive Inference

I got the rules for the LTM by inductively learning from the past training cases (like c01 to c10). I used these cases to map out how different sensor readings (like temperature, vibration, noise, pressure, and rotation) lead to specific diagnoses. These generated rules, complete with their confidence and support values, act as the static knowledge base for the whole system.

Task 3: Acquiring Short-Term Memory through Observation

Whenever I start diagnosing a new piece of equipment, the first thing the system does is run B-Prolog's `retractall/1` to totally wipe the STM and clear out any rule-firing history from previous runs. I had to do this so old facts wouldn't mess up new diagnoses. After that, it

loops through the new observations and uses assertz/1 to pop the new sensor readings into memory as dynamic facts.

To deal with missing values (like when noise=unknown), I just made it so the system ignores it and doesn't assert it into the fact base at all. This stops the pattern matching from jumping to weird conclusions just because some data is missing.

Task 4: Diagnosing Previously Unseen Situations (Inference Engine)

For the inference engine, I went with forward chaining as the main reasoning method. The core of it is an infer loop that goes through these steps:

1) Pattern Matching: It uses findall to scan the LTM and grab any rules that match the current STM but haven't been fired yet. These get dumped into a conflict set.
2) Conflict Resolution: If more than one rule can fire, I wrote a select_best logic to figure out which one goes first. It prioritizes the rule with the highest confidence. If there's a tie, it looks at the support count to break it.
3) Rule Firing & Chaining: The winning rule is marked as fired, and its conclusion gets added to the STM using assertz. This loop just keeps running until the conflict set is empty and it can't figure out anything new.

3. Advanced Considerations & Evaluation Features

To achieve a better evaluation, the system incorporates several advanced considerations:

- Explanation Capability: The system does not just provide a final diagnosis. During runtime, it explicitly prints an execution trace showing exactly which rules were fired (Fired rule: ... -> ...) alongside their confidence levels, making the reasoning process transparent.
- Handling "Unknown" Data: By carefully managing unobserved variables, the system avoids misfiring rules when sensor data is incomplete.
- Robust Conflict Resolution: The strict sorting by confidence ensures that highly reliable rules systematically override less certain intermediate conclusions, leading to a highly accurate final diagnosis.

4. B-Prolog Implementation & Step-by-Step Code Explanation

Below is the complete B-Prolog implementation of the fault diagnosis system, broken down step-by-step to explain the purpose and functionality of each logic block.

Step 1: Dynamic Predicate Declarations

```
:- dynamic fact/2.
:- dynamic fired/1.
```

Explanation: In B-Prolog, predicates are static by default. We declare fact/2 and fired/1 as "dynamic" because the system needs to constantly add (assert) new facts into the Short-

Term Memory (STM) and remove (retract) them when starting a new diagnosis. Without this, the system could not update its memory during runtime.

Step 2: Long-Term Memory (LTM) / Production Rules

```
% rule(RuleID, ConditionsList, Conclusion, Confidence, Support).
rule(learned_1, [fact(temperature, high), fact(vibration, high)],
fact(diagnosis, bearing_failure), 1.0, 3).
rule(learned_6, [fact(temperature, high), fact(pressure, low)],
fact(diagnosis, lubrication_failure), 1.0, 2).
rule(e1, [fact(temperature, high), fact(vibration, high)],
fact(machine_state, unstable), 0.9, 1).
```

Explanation: This section implements Task 1 and 2. It represents the Long-Term Memory (LTM). Each rule is a structured fact containing an ID, a list of antecedent conditions, a conclusion, a confidence value, and a support count. This acts as the knowledge base learned from past cases.

Step 3: Short-Term Memory (STM) Acquisition

```
set_observations(ObsList) :-
    retractall(fact(_, _)),  % Clear STM
    retractall(fired(_)),    % Clear triggered rules
    assert_obs(ObsList).

assert_obs([]).
assert_obs([(Attr, Val)|Rest]) :-
    assertz(fact(Attr, Val)),
    assert_obs(Rest).
```

Explanation: This fulfills Task 3. The set_observations predicate prepares the system for a new diagnosis by clearing out old facts and fired rules using retractall. Then, assert_obs recursively iterates through the new sensor readings and inserts them into the STM using assertz. If a value is "unknown", it is simply omitted from the input list, inherently preventing false inferences from missing data.

Step 4: Inference Engine (Forward Chaining)

```
infer :-
    % 1. Pattern Matching
    findall(rule(ID, Conds, Conc, Conf, Supp),
            (rule(ID, Conds, Conc, Conf, Supp),
             \+ fired(ID),
             check_conditions(Conds)),
            ConflictSet),
    ConflictSet \= [],

    % 2. Conflict Resolution
    select_best(ConflictSet, BestRule),
    BestRule = rule(BestID, _BestConds, BestConc, BestConf,
_BestSupp),
```

```
    % 3. Rule Firing
    assertz(fired(BestID)),
    ( \+ call(BestConc) -> assertz(BestConc) ; true ),

    % 4. Print Execution Trace
    write('  Fired rule: '), write(BestID), write(' -> '),
write(BestConc),
    write(' [confidence='), write(BestConf), write(']'), nl,

    % 5. Recursive Loop
    infer.
infer. % Termination condition
```

Explanation: This is the core forward-chaining reasoning loop (Task 4).
1) Pattern Matching: It uses findall to collect all rules whose conditions are met in the STM and haven't been fired yet into a ConflictSet.
2) Conflict Resolution: It calls select_best to pick the highest priority rule.
3) Rule Firing: The selected rule is marked as fired, and its conclusion is added to the STM.
4) Explanation: It prints the reasoning trace. The loop repeats continuously until no new rules can fire (the conflict set is empty), triggering the termination condition.

Step 5: Helper Functions (Condition Checking & Conflict Resolution)

```
check_conditions([]).
check_conditions([Cond|Rest]) :- call(Cond), check_conditions(Rest).

select_best([Rule], Rule).
select_best([rule(ID, Cond, Conc, Conf, Supp) | Rest], BestRule) :-
    select_best(Rest, rule(ID2, Cond2, Conc2, Conf2, Supp2)),
    ( (Conf > Conf2 ; (Conf == Conf2, Supp >= Supp2)) ->
      BestRule = rule(ID, Cond, Conc, Conf, Supp)
    ;
      BestRule = rule(ID2, Cond2, Conc2, Conf2, Supp2)
    ).
```

Explanation: check_conditions is a recursive helper that ensures every condition in a rule's antecedent exists in the current STM. select_best implements the conflict resolution strategy: it compares rules in the conflict set, strictly prioritizing the Confidence value, and uses

Support as a secondary tie-breaker if confidences are equal.

